

DevOps & Cloud Engineering for AI Applications

Level instructions and case study — Manager and Senior Manager, Platform/DevOps Engineering

This exercise evaluates DevOps and cloud/infrastructure engineering skill using a generative-AI application as the workload. It intentionally keeps the AI/model logic simple so your time and our evaluation stay focused on build, deployment, and operability — the skills the role is actually hiring for.

Reference architectures and starter repositories

You are not expected to build from a blank folder, and you do not need to use any of these as-is — fork, strip down, or simply borrow the parts (folder structure, CDK stack layout, pipeline YAML) that are useful. Picking a slice of a larger reference and explaining what you kept and what you cut is itself a signal we look for.

Repository / Resource	What it demonstrates	How to use it here
aws-samples/aws-genai-llm-chatbot	Production-grade reference for a multi-LLM, multi-RAG chatbot on AWS CDK: separate stacks for auth, API, RAG engines, and model access; CI workflows included.	Best source for how to modularize a real GenAI platform into CDK stacks. Pick 2–3 stacks (e.g. RAG engine + API) rather than deploying the whole solution.
awslabs/generative-ai-cdk-constructs	Official, well-architected CDK L2/L3 constructs for Bedrock, Knowledge Bases, and vector stores — patterns rather than a full app.	Strong reference for Case 2 (IaC). Shows what “production-ready” GenAI infrastructure code looks like at the module level.
aws-samples/sample-bedrock-serverless-react-chatbot	Smaller, single-repo serverless Bedrock chatbot with a scripted CDK deploy, optional RAG, and a clear on/off toggle for infrastructure pieces.	A good size reference for a one-week exercise — notice how it lets you skip RAG infra entirely to control scope.
aws-samples/Serverless-Retrieval-Augmented-Generation-RAG-on-AWS	Full-stack serverless RAG built for prototypes/MVPs, using S3 + LanceDB as a low-cost vector store.	Useful if you want to avoid running a managed vector database at all — shows a cheaper, file-based alternative worth mentioning as a trade-off in your presentation.
aws-containers/retail-store-sample-app	The original microservice sample (no AI component) — still a solid reference purely for container/CI/CD mechanics.	If you'd rather keep the AI logic trivial (e.g. a single Lambda calling Bedrock) and focus your engineering effort on pipeline/IaC quality, you can borrow this app's service structure for the non-AI parts.

Optional: sample corpus / workload data

If your case needs a document corpus to index (for the RAG ingestion job) or a dataset to justify the data layer in your IaC, a small Kaggle dataset is an easy, realistic stand-in. Keep it small (a few MB, a few hundred records) — the point is exercising the pipeline, not the data itself.

Kaggle dataset	Why it fits	Suggested use
Amazon Fine Food Reviews	Realistic short-text corpus, easy to chunk and embed, large enough to subsample to any size you want.	Sample 200–500 reviews as the knowledge base for a RAG Q&A service (“ask questions about product feedback”).
arXiv Dataset (metadata + abstracts)	Structured abstracts with titles/categories — good for a “research assistant” style RAG demo with clear ground-truth questions.	Filter to one category (e.g. cs.AI) and a recent date range to keep the corpus small before ingestion.
Wikipedia Movie Plots	Clean, self-contained plot summaries — zero domain expertise needed to write test questions and eyeball answer quality.	Good default if you want something low-effort to reason about while you focus your time on pipeline/infra quality.

Senior Manager Platform/DevOps Engineer

A note on AI usage: Using AI assistants (coding assistants, chat tools, etc.) for this exercise is allowed and encouraged. We want to see how you actually work, and that includes how effectively you use the tools available to you. The only requirement is transparency — please be ready to clearly disclose to the interviewers which parts of your solution were AI-assisted and how you used it.

At the Senior Manager level, we expect a solution that demonstrates strong hands-on platform engineering capability combined with product-minded software delivery. You should focus on building a robust, maintainable, and production-ready solution that reflects how you would deliver build and infrastructure capabilities for an AI-powered product in a real environment.

We expect you to approach the case not only as a build or provisioning exercise, but as an end-to-end engineering problem: from pipeline and infrastructure design to deployment strategy, operability, and clear technical communication. Your solution should show sound engineering judgment, practical trade-off decisions, and strong attention to reliability, cost, and operational quality specific to running a generative-AI workload.

In addition to technical depth, we are looking for evidence that you can work effectively in a cross-functional product setting, balancing speed, reliability, security, and maintainability. We also value how you structure your work so it can be extended by others, including through clear documentation, modular design, and good engineering practices.

What we mean by “AI application” in this exercise

The workload is a **generative-AI service** — typically a retrieval-augmented generation (RAG) API or chatbot backend — not a model-training pipeline. You are not being asked to train a model.

Use a **managed foundation model** (Amazon Bedrock is the easiest path — no GPU quota requests, pay-per-token) rather than self-hosting an LLM, unless you specifically want to showcase GPU/inference infrastructure, in which case a small open-weight model on SageMaker or Fargate is a reasonable “extra credit” alternative.

A realistic AI service for this exercise has three moving parts you can reason about independently: an **ingestion/indexing job** (turns a small document set into embeddings), a **vector store** (OpenSearch Serverless, Aurora pgvector, or similar), and an **inference API** (retrieves context, calls the model, returns an answer). Treat each as a component with its own build, deploy, and rollback story.

Case study

Please choose one case you're more comfortable with for your case study. Your delivery should include:

- Code repository (either Git repo or Zip)
- Presentation, either as a PowerPoint file or as a self-contained HTML page

Optional

- Architecture diagram covering the AI service end to end — ingestion/indexing, vector store, inference API, and how the pipeline or IaC deploys each piece

Disclosure

We understand that this is a comprehensive request and may be challenging to complete within one week.

We encourage you to prioritize the most critical aspects of the task, such as demonstrating your problem-solving approach, technical skills, and ability to communicate results effectively.

Summarize your findings and results in a presentation (PowerPoint or HTML) with a maximum of 10 slides/sections. Focus on clarity, conciseness, and how your solution would work in a realistic production context.

As noted above, using an AI coding assistant is welcome. Make sure you disclose the usage of it to the interviewers, as well as how you used it to develop your solution.

Good luck, and we look forward to reviewing your work!

Case 1: CI/CD Pipeline for a GenAI RAG Microservice on AWS (Build & Release)

Objective:

Design and implement a CI/CD pipeline that builds, tests, and deploys a small retrieval-augmented generation (RAG) service to AWS: an ingestion job that embeds a small document set, and an inference API that answers questions using that index and a foundation model. Use one of the reference repos above as a starting point or scope reference — feel free to strip it down to 2–3 components rather than the full solution.

Task:

Build a pipeline that automates linting, testing, building container images, and deploying your chosen components to AWS using a simple, lightweight setup (e.g., ECS or App Runner). Apply appropriate practices for versioning, artifact management, and promoting a build across at least two environments. In addition to pipeline mechanics, show how you would structure the solution for production use, including a deployment strategy (e.g., blue-green or canary) for the inference API, a rollback approach for both the service and the vector index, and basic release monitoring or gating — for example, a response-quality or latency check gating promotion to production.

Case 2: Infrastructure as Code for a GenAI/RAG Platform on AWS (Infra/DevOps)

Objective:

Provision the AWS infrastructure needed to run a small GenAI RAG service across two or more environments: a vector store, a foundation model access path, and the compute/networking for the inference API. Keep the footprint simple: a basic networking setup, a compute layer, and a managed vector store are sufficient.

Task:

Write Infrastructure as Code to provision the environments in a reproducible, parameterized way. The choice of IaC tool is entirely up to you (e.g., Terraform, AWS CDK, CloudFormation, or Pulumi). Handle secrets and model/API access cleanly, structure the code into reusable modules, and evaluate your design using criteria such as repeatability and blast-radius containment. In addition to provisioning, demonstrate how you would operate the environment in practice, including basic monitoring/alerting

(e.g., token usage, latency, error rate on the inference path), a rollback or disaster-recovery approach for the vector index, and any policy-as-code or guardrails you would apply — such as IAM scoping to specific models, request throttling, or cost budgets/alerts.

Evaluation Criteria

- Platform Engineering: Clean, modular, well-structured implementation that connects build, infrastructure, and operability in a product-oriented way.
- Software Engineering Quality: Strong code organization, maintainability, testing approach, version control discipline, and engineering best practices.
- Infrastructure & Pipeline Design: Appropriate tool selection, thoughtful modularization, sound evaluation, and clear explanation of trade-offs.
- Production Readiness: Ability to operationalize the solution through deployment strategy, rollback/DR approach, monitoring considerations, and reproducibility.
- AI Platform Judgment: Evidence of reasoning specific to generative-AI workloads — model access strategy, cost/throttling controls, index lifecycle management, and response-quality gating — rather than generic infra practices applied without adaptation.
- Security & Reliability: Evidence that secrets, access, and failure modes were considered, with reasonable guardrails in place.
- Technical Communication & Judgment: Clear explanation of design choices, assumptions, constraints, and prioritization decisions.
- Collaboration & Team Contribution Mindset: Signals that the solution is designed in a way that supports extensibility, knowledge sharing, and collaboration across engineering teams.

Deliverables

- Full codebase organized with modular architecture (for example, separate modules for pipeline definitions, infrastructure modules, and application/build scripts).
- A working CI/CD pipeline and/or IaC module set that provisions and deploys the solution end to end.
- Clear documentation describing project structure, setup steps, assumptions, model/vector-store choices, and how to run or test the solution.
- A concise presentation (PowerPoint or HTML) summarizing the problem, approach, technical decisions, results, and how the solution could be operated in production.
- Optional: Kubernetes manifests or Helm chart, a basic monitoring/observability dashboard, or a cost/scaling analysis specific to the inference workload.

Appendix: guidance, scope, and common pitfalls

This section applies to both levels and both cases. It exists to help you spend your limited time on the things we actually evaluate.

What “good” looks like at a glance

- A pipeline or IaC module set that actually runs, even if the AI logic behind it is intentionally simple.
- Clear reasoning for trade-offs (managed vs. self-hosted model, vector store choice, blast-radius containment) rather than defaulting to the fanciest option.
- Evidence you thought about the AI-specific failure modes: a bad or slow model response, a stale index, a runaway cost from an unthrottled endpoint — not just generic infra failure modes.

Common pitfalls to avoid

- Spending most of the week on the AI/model logic and leaving little time for the pipeline or IaC quality — remember the AI service is the workload, not the subject of evaluation.
- Deploying the full reference repo unmodified — we want to see your own structuring and judgment on a smaller slice, not a fork with your name on it.
- Skipping the second environment or the rollback story — these are explicitly scored and easy to shortchange under time pressure.